

S12. React Router, useReducer and Redux Toolkit

Introduction

Modern React provides developers with a rich ecosystem of tools for managing navigation and application state in web apps. In building single-page applications (SPAs), three fundamental challenges arise: navigation between views without full page reloads, managing complex local component state, and sharing global state across the component tree.

This document covers:

- Navigation principles with React Router v6 and its new APIs compared to previous versions.
- Managing complex local state with useReducer, compared to global state management with Redux.
- Introduction to Redux Toolkit (RTK) as the modern, simplified way to use Redux.
- Handling asynchronous actions with Redux Thunk middleware, specifically using createAsyncThunk from RTK.

Examples throughout this document are inspired by a notes management application, illustrating the practical application of each concept.

REACT ROUTER v6 — Navigation in a SPA

React Router is the standard library for managing navigation in a React single-page application. It enables client-side routing — defining routes that map URL paths to React components, allowing navigation without full page reloads. The browser's History API is used internally to update the URL without triggering a server request.

Core components in React Router v6:

- **BrowserRouter:** wraps the entire React application and monitors the browser's navigation history. Must appear once at the top level.
- **Routes:** container for all route definitions. Replaces the old Switch from v5. Renders only the first matching route.
- **Route:** maps a URL path to a React component. Attributes: path (URL pattern) and element (the React component to render).
- **Link / NavLink:** renders anchor tags that navigate without page reload. NavLink adds an active class when the route matches.
- **useNavigate:** hook for programmatic navigation. Replaces useHistory from v5.
- **useParams:** hook for reading dynamic route parameters (e.g. /books/:id).
- **Outlet:** placeholder in a parent route component where nested child routes are rendered.

Differences from React Router v5:

- Switch becomes Routes: Routes renders only the first matching route, eliminating the need for the exact prop.
- component prop replaced by element: v6 uses element={} instead of component={Component}.
- Exact matching by default: routes match exactly without needing the exact attribute.
- useHistory replaced by useNavigate: cleaner API for programmatic navigation.
- Native nested routes support: layouts with shared headers and footers are easier to implement.

v5 vs v6 comparison:

```
// React Router v5
// React Router v6 } /> } />
```

Nested routes example:

MainLayout renders shared UI elements (header, navigation, footer) and a component where child route components are injected. This eliminates the need to repeat layout code in every page component.

Adding a new route — practical example:

```
// Adding an /about page } /> } /> // Navigation link in the header About
```

Best practice: ensure BrowserRouter wraps the application only once at the top level. Never use regular HTML anchor tags for internal navigation — always use Link or NavLink to avoid full page reloads.

MANAGING LOCAL STATE with useReducer

In React, every component can manage its own state using hooks like `useState` or `useReducer`. When state update logic becomes complex, `useReducer` provides a more structured alternative. It follows the same pattern as `Redux` — a reducer function takes the current state and an action and returns the new state.

When to use useReducer:

- State has complex structure (objects with multiple fields, nested data, arrays).
- State transitions are non-trivial or interdependent — multiple fields change together.
- Updates are logically grouped operations (increment/decrement, reset, add to list).
- Managing all fields with multiple `useState` calls would be unwieldy.

useReducer vs Redux:

The key difference is scope. `useReducer` is ideal for complex local state within a single component or a small subtree. `Redux` manages global state shared across many components that are not directly related in the component hierarchy.

Example — useReducer for a note form:

```
import { useReducer } from 'react'; const initialState = { title: '', content: '' }; function formReducer(state, action) { switch (action.type) { case 'SET_TITLE': return { ...state, title: action.payload }; case 'SET_CONTENT': return { ...state, content: action.payload }; case 'RESET': return initialState; default: return state; } } function NoteForm() { const [formState, dispatch] = useReducer(formReducer, initialState); return ( <div> <input type="text" value={formState.title} /> <input type="text" value={formState.content} /> <button type="button" value="Clear" /> </div> </form> ) </function>
```

All form state logic is centralised in `formReducer`. Adding a new field or a new operation requires only updating the reducer and adding a new action type — the component itself remains clean.

Note: if other parts of the application need access to this form's data, consider lifting state up to a common ancestor or moving it to a global `Redux` store.

GLOBAL STATE WITH REDUX — Principles and Configuration

As applications grow, the need for global state arises — data that must be accessed and modified by components at different levels of the component tree, with no direct parent-child relationship. Redux provides a predictable, centralised state container for JavaScript applications.

Core Redux principles:

- Single source of truth: one global store holds the entire application state as an object tree.
- State is read-only: the only way to change state is to dispatch an action — a plain object describing what happened.
- Pure reducer functions: reducers take current state and an action and return a new state, with no side effects.
- Unidirectional data flow: action dispatched → reducer processes → new state produced → components re-render.

Redux Toolkit (RTK) — the modern approach:

Redux Toolkit is the official recommended way to write Redux logic. It simplifies store setup, reduces boilerplate, and includes powerful utilities that would otherwise require multiple separate packages.

- `configureStore`: creates a pre-configured Redux store with best practices built in. Automatically combines reducers, enables Redux DevTools, and adds Thunk middleware.
- `createSlice`: defines a slice of global state with its initial value, reducers, and automatically generated action creators in one call.
- Immer integration: RTK includes Immer under the hood, allowing reducer logic to be written as if mutating state directly — Immer handles immutable updates internally.
- `createAsyncThunk`: utility for handling async operations with automatic pending/fulfilled/rejected action dispatching.

Example — defining a notes slice:

```
// store/notesSlice.js import { createSlice } from '@reduxjs/toolkit'; const
notesSlice = createSlice({ name: 'notes', initialState: [ { id: 1, content: 'First
note' }, { id: 2, content: 'Second note' }, ], reducers: { addNote: (state, action)
=> { // Immer allows this 'mutation' syntax – actually immutable state.push({ id:
Date.now(), content: action.payload }); }, deleteNote: (state, action) => { return
state.filter(note => note.id !== action.payload); }, }, }); export const { addNote,
deleteNote } = notesSlice.actions; export default notesSlice.reducer;
```

Configuring the global store:

```
// store/index.js import { configureStore } from '@reduxjs/toolkit'; import
notesReducer from './notesSlice'; export const store = configureStore({ reducer: {
notes: notesReducer, }, });
```

Providing the store to the React app:

```
import { Provider } from 'react-redux'; import { store } from './store';
ReactDOM.render( , document.getElementById('root') );
```

Using Redux state and actions in components:

```
import { useSelector, useDispatch } from 'react-redux'; import { deleteNote } from
'./store/notesSlice'; function NotesList() { const notes = useSelector(state =>
state.notes); const dispatch = useDispatch(); return ( {notes.map(note => (
{note.content} dispatch(deleteNote(note.id))>Delete ))} ); }
```

useSelector reads data from the store. useDispatch returns the dispatch function for sending actions. Components re-render automatically when the selected state slice changes.

ASYNC ACTIONS with Redux Thunk and createAsyncThunk

Real applications need to fetch data from APIs, save to databases, and handle loading and error states. Redux reducers must be pure synchronous functions — they cannot contain async logic or side effects. The solution is middleware: code placed between the action dispatch and the reducer that can intercept actions, execute async operations, and dispatch new actions based on results.

Redux Thunk middleware:

Redux Thunk allows action creators to return functions instead of plain objects. These functions receive dispatch and getState as arguments and can perform async operations before dispatching actions. Redux Toolkit includes Thunk middleware by default — no additional configuration is required.

createAsyncThunk:

createAsyncThunk is RTK's utility for handling async operations with automatic lifecycle management. When called, it automatically dispatches three actions:

- prefix/pending: dispatched immediately when the async function starts.
- prefix/fulfilled: dispatched when the promise resolves successfully, with the returned value as payload.
- prefix/rejected: dispatched when the promise throws an error, with error details as payload.

Example — async delete with createAsyncThunk:

```
// store/notesSlice.js (extended) import { createAsyncThunk, createSlice } from
 '@reduxjs/toolkit'; // Define async thunk for deleting a note via API export const
 deleteNoteAsync = createAsyncThunk( 'notes/deleteNoteAsync', async (noteId) => {
  await fetch(`/api/notes/${noteId}`, { method: 'DELETE' }); return noteId; //
  returned value becomes action.payload in fulfilled } ); const notesSlice =
 createSlice({ name: 'notes', initialState: { items: [], loading: false, error: null
 }, reducers: {}, extraReducers: (builder) => { builder
  .addCase(deleteNoteAsync.pending, (state) => { state.loading = true; state.error =
  null; }) .addCase(deleteNoteAsync.fulfilled, (state, action) => { state.loading =
  false; state.items = state.items.filter(n => n.id !== action.payload); })
  .addCase(deleteNoteAsync.rejected, (state, action) => { state.loading = false;
  state.error = action.error.message; }); }, });
```

Using the async thunk in a component:

```
function NotesList() { const { items, loading, error } = useSelector(state =>
 state.notes); const dispatch = useDispatch(); if (loading) return Deleting...; if
 (error) return Error: {error}; return ( {items.map(note => ( {note.content}
 dispatch(deleteNoteAsync(note.id))> Delete ))} ); }
```

REDUX TOOLKIT vs CLASSIC REDUX — Comparison

Aspect	Classic Redux	Redux Toolkit
Store setup	Manual combineReducers + createStore	configureStore — one call, best practices built in
Boilerplate	Action types, action creators, reducers	createSlice generates all three together
Immutability	Must return new objects manually	Immer allows direct mutation syntax
Async actions	Separate redux-thunk install + manual call	createAsyncThunk built in, lifecycle automatic
DevTools	Manual enhancer configuration	Enabled automatically by configureStore
Code volume	High — many files for simple features	Low — slice contains everything for one feature
Learning curve	Steep — many concepts to combine	Gentler — guided by RTK conventions

Redux Toolkit is the officially recommended approach for all new Redux projects. It reduces the number of files required to implement a feature by approximately 60%, while maintaining full compatibility with the Redux DevTools and the broader ecosystem.

SUMMARY

- React Router v6 manages client-side navigation with BrowserRouter, Routes, and Route. Key hooks: useNavigate, useParams. Key change from v5: Routes replaces Switch, element replaces component.
- useReducer manages complex local component state with a reducer function. Best for components with multiple interdependent state fields. Not suitable for state shared across unrelated components.
- Redux provides a global, centralised state store. Core principles: single store, read-only state, pure reducers, unidirectional data flow.
- Redux Toolkit simplifies Redux with configureStore, createSlice, and createAsyncThunk. Includes Immer for simplified immutable updates and Thunk middleware by default.
- createAsyncThunk handles async operations with automatic pending/fulfilled/rejected lifecycle actions. Components handle all three states for proper loading and error UI.

REVIEW QUESTIONS

- What is the difference between BrowserRouter and Routes in React Router v6?
 - When would you choose useReducer over useState for local state management?
 - What are the three core principles of Redux?
 - What does createSlice generate automatically?
 - What are the three lifecycle actions dispatched by createAsyncThunk?
 - What is the difference between useSelector and useDispatch?
 - When should you use Redux instead of lifting state up with useReducer?
 - What problem does Immer solve in Redux Toolkit reducers?
-

BIBLIOGRAPHY

React Router. (n.d.). React Router v6 Documentation. <https://reactrouter.com/>

Redux Toolkit. (n.d.). Redux Toolkit Official Documentation. <https://redux-toolkit.js.org/>

React. (n.d.). useReducer Hook. <https://react.dev/reference/react/useReducer>

Chamseddine, H. (2024). Taming React State: A Guide to useReducer. DEV Community.

Hanson, M., & Larry, J. (2023). Redux Toolkit in Practice. Packt Publishing.

Salguera, R. (2022). React Router v6: The Complete Migration Guide. Medium.

Facebook/Meta. (n.d.). React Documentation. <https://react.dev/>